VEGA

Verify · Execute · Guide · Audit

PS-03 | Runbook-Following Agent

Use Case & System Diagrams Report

Plan-and-Execute Agent + Custom MCP Shell Tool (Allowlisted)

Stack: React (JS) · Python · SQLite · Ollama (Local LLM)

1. System Actors & Use Cases

1.1 Actors

Actor	Description
On-Call Engineer	Engineer paged at 2 AM; monitors VEGA, approves HIGH-risk steps, can abort.
Incident Source	Alertmanager / PagerDuty / monitoring webhook posting the JSON alert.
Runbook Repository	Git-backed folder of Markdown runbooks indexed by alert type.
Planner Agent (Ollama)	Local LLM that parses the runbook and tags each step with a risk level.
Executor Agent	Python dispatcher that routes each step to the correct adapter.
MCP Shell Tool	Allowlist-enforced subprocess executor with timeout and capture.
Target Systems	Hosts (Shell), REST APIs and Databases (SQL) that VEGA operates on.
Audit Log Store	Append-only SQLite table holding a hash-chained record of every action.

1.2 Use Case Summary

UC#	Use Case	Primary Actor	Brief Description
UC1	Ingest Alert & Load Runbook	Incident Source	Match alert to runbook, create session in SQLite.
UC2	Parse Numbered Steps	Planner Agent	Tokenise Markdown into ordered, structured steps.
UC3	Verify Step & Classify Risk	Planner Agent	LLM + rules tag each step LOW / MEDIUM / HIGH.
UC4	Execute Safe Step via MCP Shell	Executor Agent	Run via allowlisted subprocess with timeout.
UC5	Guide — Request Human Confirmation	On-Call Engineer	Pause on HIGH-risk; engineer approves or rejects.
UC6	Handle Multi-Input Step	Executor Agent	Shell / REST / SQL / File (HTML/CSV/JSON) adapters.
UC7	Enforce Allowlist	MCP Shell Tool	Hard-deny any command outside YAML allowlist.
UC8	Stream Live Log to UI	On-Call Engineer	SSE stream of stdout/stderr to React console.
UC9	Append Audit Record	Audit Log Store	Hash-chained, immutable row per action.

2. Use Case Diagram

This diagram captures all external actors and the nine use cases within the VEGA system boundary, including the «includes» and «extends» relationships between them.

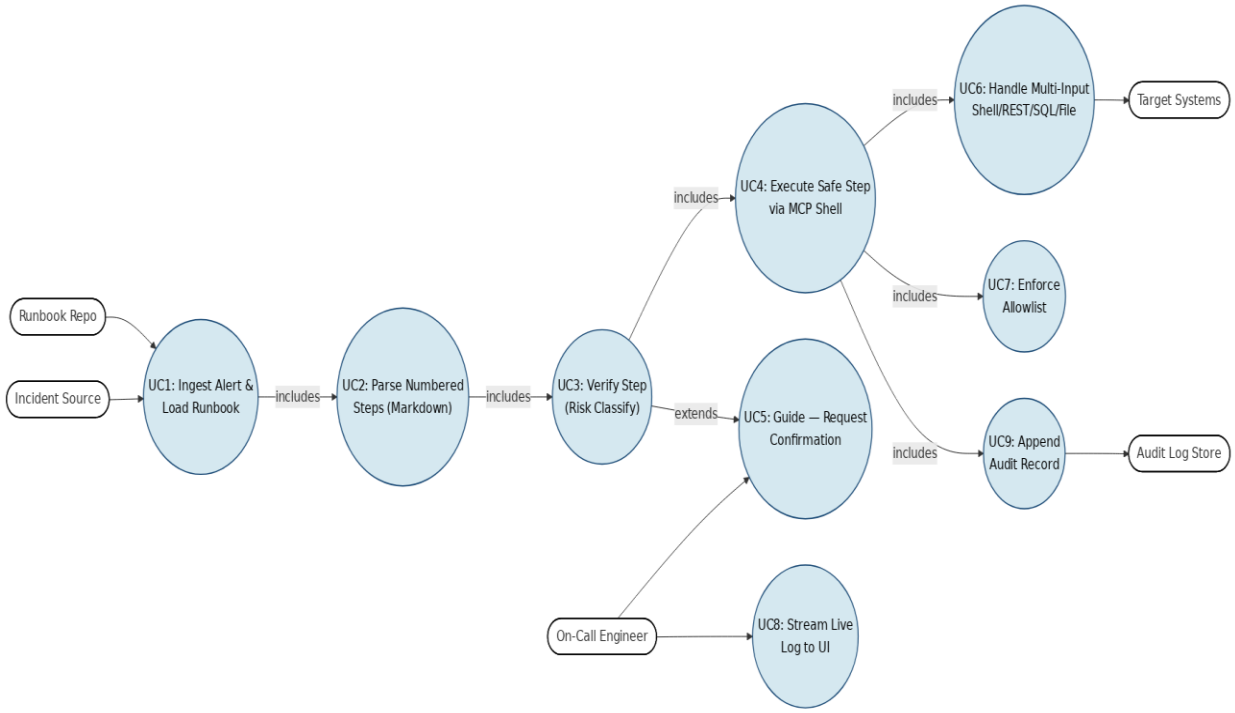

Figure 1 — Use Case Diagram. All nine use cases mapped to their primary actors with includes / extends relationships.

3. Sequence Diagram

The sequence diagram captures the full message lifecycle for a single alert — from JSON ingestion through risk classification, MCP shell execution, human-in-the-loop approval on HIGH-risk steps, audit persistence and live log streaming back to the operator.

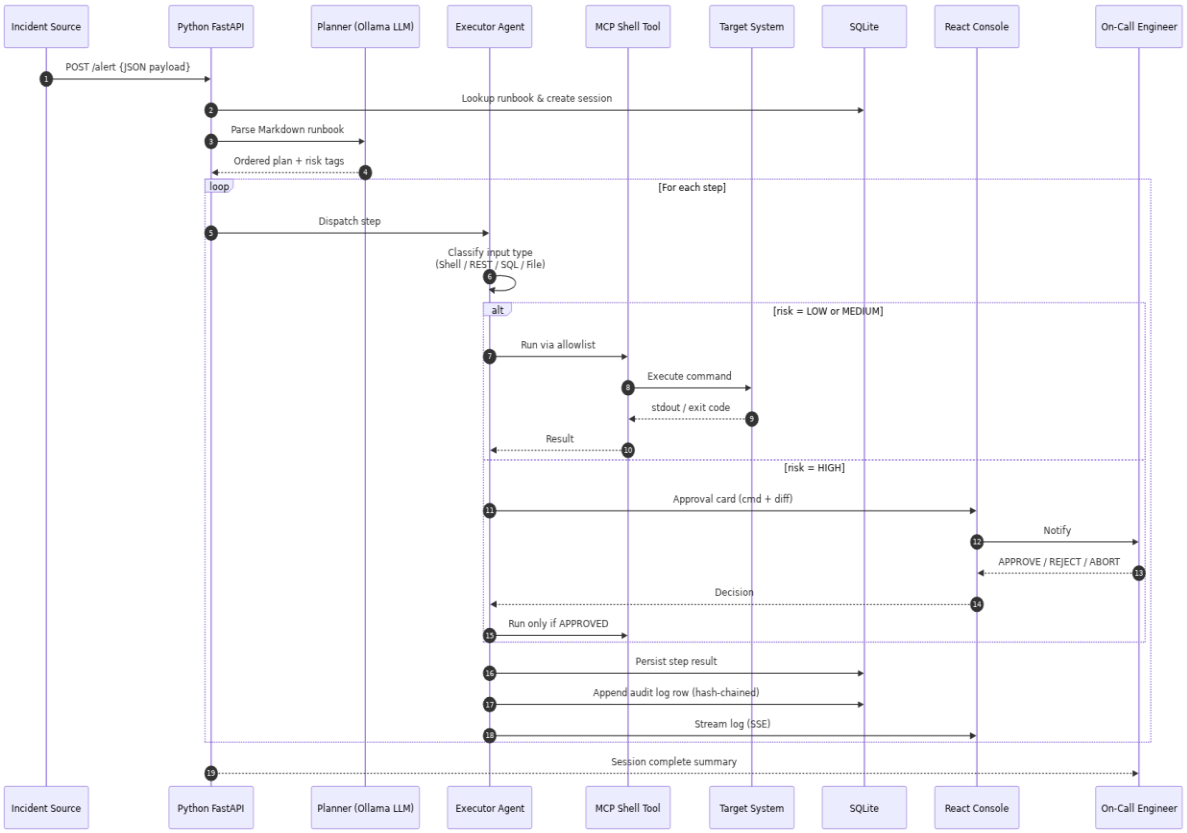

Figure 2 — Sequence Diagram. Full lifecycle of one runbook execution session.

4. Activity / Flowchart Diagram

The activity diagram models VEGA's decision logic — input-type classification, allowlist enforcement, risk-tier branching, retry/escalate on failure, and audit logging on every path.

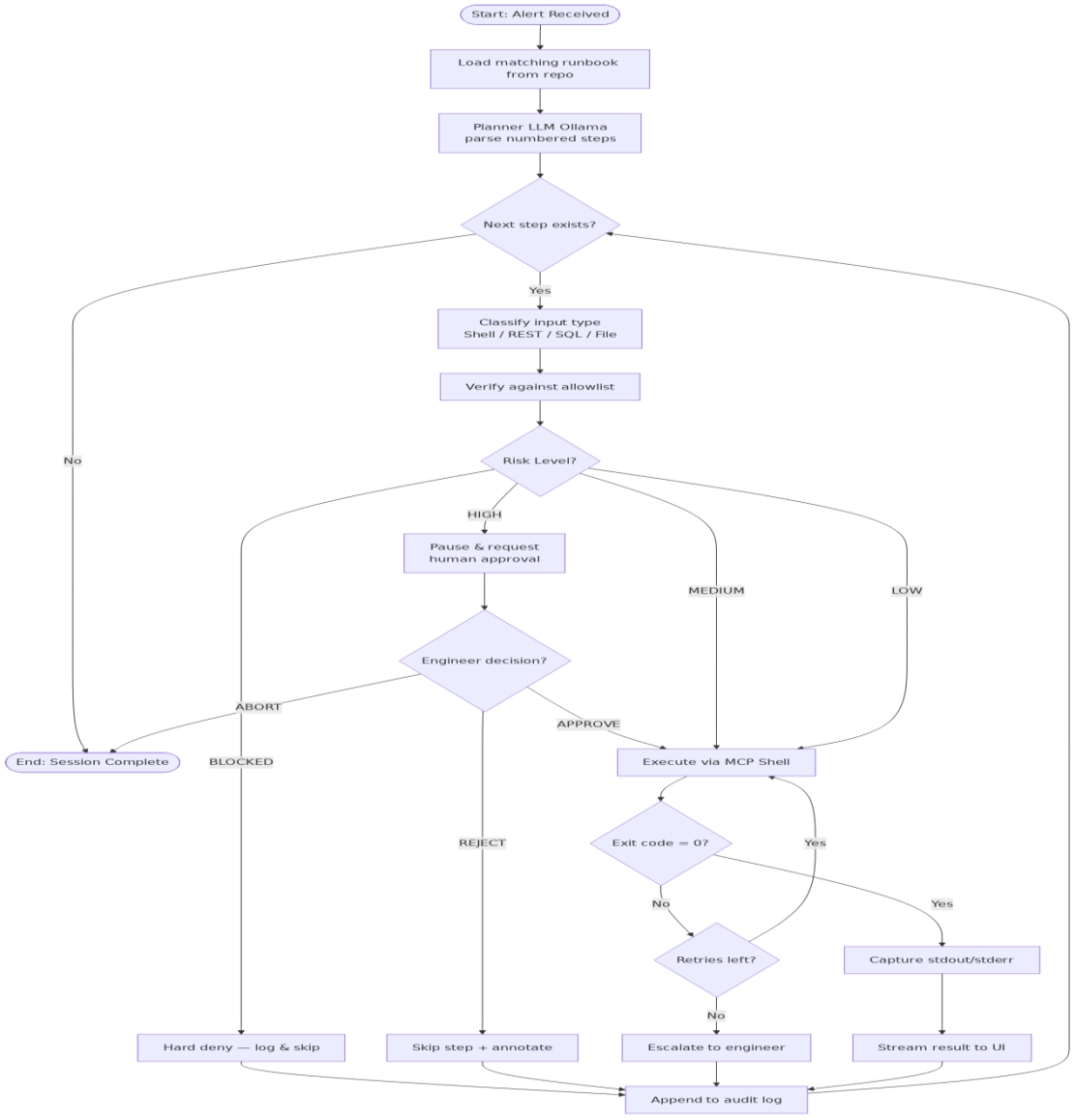

Figure 3 — Activity Diagram. Decision logic for a single runbook step.

5. System Architecture Diagram

VEGA is organised in seven layers from React UI down to target systems. The MCP Shell allowlist sits between the executor and any subprocess so no command can reach the OS unchecked.

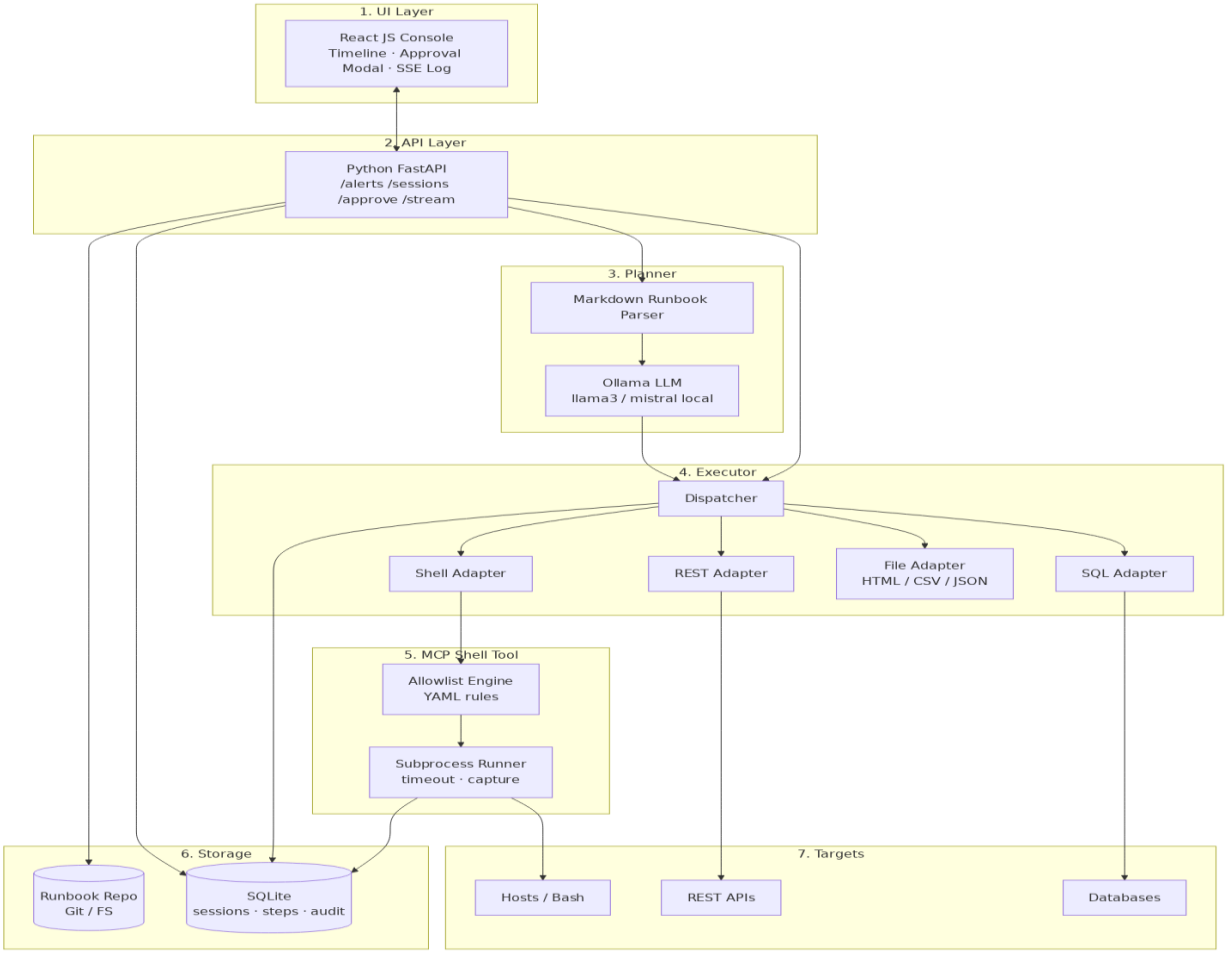

Figure 4 — System Architecture. React (JS) UI · Python FastAPI · Ollama planner · MCP Shell with allowlist · SQLite audit store.

5.1 Layer Summary

#	Layer	Key Components / Technology
1	UI Layer	React (JavaScript) console — runbook timeline, approval modal, SSE log stream.
2	API Layer	Python FastAPI — /alerts, /sessions, /approve, /stream endpoints.
3	Planner	Ollama local LLM (llama3 / mistral) + Markdown runbook parser.
4	Executor	Python dispatcher with Shell / REST / SQL / File adapters.
5	MCP Shell Tool	YAML allowlist engine + subprocess runner with timeout & capture.
6	Storage	Git-backed runbook repo + SQLite (sessions, steps, audit log).
7	Targets	Bash hosts, REST APIs, SQL databases — the systems VEGA operates on.

6. System Inputs

VEGA accepts heterogeneous inputs through dedicated executor adapters:

- Shell / Bash — executed via the MCP shell wrapper against the YAML allowlist.
- REST API — HTTP GET/POST with JSON, used for health probes and restart endpoints.
- SQL — read-only diagnostic queries against SQLite or target databases.
- Files — Markdown runbooks (.md), HTML pages, CSV metric exports, JSON configs and alert payloads.

7. MCP Shell Tool — Allowlist Tiers

Risk	Behaviour	Examples
LOW	Auto-execute, log only	ls, cat, grep, ps, df -h, systemctl status, curl -I
MEDIUM	Auto-execute with strict limits	systemctl restart <svc>, kubectl rollout status, tail -n 200
HIGH	Block until human approval	systemctl stop, kubectl delete, DROP / TRUNCATE / DELETE, rm -rf
BLOCKED	Never executed (hard deny)	mkfs, dd of=/dev/*, shutdown, fork bombs

8. Use Case Specifications

8.1 UC1 — Ingest Alert & Load Runbook

Field	Detail
Primary Actor	Incident Source (Alertmanager / PagerDuty)
Precondition	FastAPI alert webhook is reachable; runbook repo is mounted.
Main Flow	POST /api/alerts → validate JSON → match alert.name to runbook → create session row → return session_id.
Exception	No matching runbook → 404; on-call engineer paged directly.
Postcondition	Session created in SQLite; planner enqueued.

8.2 UC3 — Verify Step & Classify Risk

Field	Detail
Primary Actor	Planner Agent (Ollama)
Input	Numbered step text from the Markdown runbook.
Output	{ command, input_type ∈{shell,rest,sql,file}, risk ∈{LOW,MED,HIGH}, rationale }.
Rule	Any destructive verb (delete, drop, stop, restart prod) → HIGH regardless of LLM output.
Postcondition	Step row updated with risk tag and rationale.

8.3 UC4 — Execute Safe Step via MCP Shell

Field	Detail
Primary Actor	Executor Agent
Precondition	risk ∈{LOW, MEDIUM} AND command passes allowlist.
Main Flow	Dispatch to adapter → run with timeout (default 30s) → capture stdout/stderr/exit code → persist.
Alt Flow	Exit ≠ 0 → retry up to N times → escalate to engineer.
Postcondition	Result streamed via SSE; audit row appended.

8.4 UC5 — Guide: Request Human Confirmation

Field	Detail
Primary Actor	On-Call Engineer
Trigger	risk = HIGH OR allowlist miss.
Main Flow	Pause session → push approval card (command, diff, blast radius) to React UI.
Decisions	APPROVE → execute; REJECT → skip + annotate; ABORT → terminate session.
Postcondition	Decision recorded with approver identity and timestamp.

8.5 UC7 — Enforce Allowlist

Field	Detail
Primary Actor	MCP Shell Tool
Allowlist	YAML file: { binary, allowed_flags[], forbidden_args[] }.
Behaviour	Hard deny — request never reaches subprocess; event logged as BLOCKED.
Postcondition	Engineer notified on any deny event in real time.

9. Risks & Mitigations

Risk	Mitigation
LLM hallucinates a destructive command	Hard allowlist + destructive-verb rule overrides LLM risk tag.
Runbook drifts from production reality	Dry-run mode; engineer can edit step before approval.
Approval fatigue at 2 AM	Risk tiers — LOW/MED auto-execute, engineer sees only HIGH steps.
Audit log tampering	Append-only SQLite with per-row SHA-256 chained to previous row.
Ollama unavailable	Fallback to rule-based parser (regex on numbered steps + keyword tags).

10. Success Metrics

- MTTR for runbook-driven incidents reduced by ≥ 40%.
- 0 unauthorised destructive commands executed (allowlist bypass = 0).
- ≥ 95% of LOW / MEDIUM steps auto-executed without human input.
- 100% of HIGH steps gated by a recorded human approval.
- 100% of executed steps present in the immutable audit log.